PRESTIX.VIP

Addendum B

Multi-Promoter Preference Attribution
& Venue Promoter Performance Dashboard

This addendum extends the Magic Link & Referral Implementation Strategy (v1.0) with two new specification domains:

- Part A — Multi-Promoter Preference: the rules, UX, and data model governing a member who holds active referral relationships with more than one promoter for the same venue on the same event date, and how their 'favourite' (bound) promoter is selected and locked for the duration of that event's expenditure.
- Part B — Venue Promoter Performance Dashboard: the metrics, calculation methods, schema, and UI surface that allow a Venue Admin to evaluate, rank, and select promoters based on historical performance data across conversion, velocity, pre-event, on-night, and general spending dimensions.

Attribute	Value
Document type	Technical Specification Addendum
Extends	PRESTIX Magic Link & Referral Implementation Strategy v1.0
Scope	MEMBER role (Part A) · VENUE_ADMIN role (Part B)
New models	MemberPromoterPreference, PromoterVenuePerformance, PromoterPerformanceSnapshot
New enums	PromoterBindingScope, PromoterPerformanceDimension
Version	1.0 — March 2026

Part A — Multi-Promoter Preference Attribution

The base attribution model assumes a member has a single attributed promoter at any time. This part defines what happens when a member legitimately holds multiple active promoter relationships — for example, they received links from two different promoters both promoting the same venue and event — and how the system resolves which promoter receives commission for any given transaction.

A.1 When Multi-Promoter Conflict Arises

A conflict arises when ALL of the following conditions are simultaneously true:

1. A member has clicked and attributed to Promoter A's magic link (attribution established, decayCounter = 0).
2. The same member subsequently clicks and attributes to Promoter B's magic link, where Promoter B's link is scoped to the SAME venue and overlapping event date window as Promoter A's original attribution.
3. The member has not yet completed any transaction that would canonically resolve the attribution (i.e., no booking has been made to trigger the decay model).

> **Why this only triggers at the venue+date intersection**
>
> Two promoters promoting entirely different venues or non-overlapping dates do not create a conflict — the member simply has separate attributions per venue. The conflict is scoped: a member may support Promoter A at Venue X and Promoter B at Venue Y simultaneously with no issue. Only when both promoters are targeting the same venue+event+date does a preference resolution become necessary.

A.2 Resolution Flow — Member Favourite Selection

When a second attribution conflict is detected at the time of link ingestion (middleware layer), the system does NOT silently override. Instead, the member is presented with a Promoter Preference prompt before proceeding to the booking flow. The flow is:

4. Member taps Promoter B's link. Middleware validates the link and detects an existing active attribution for the same venue+date scope (Promoter A).
5. Rather than overriding, the system stores the new candidate attribution as a pending MemberPromoterPreference record with status = PENDING_CHOICE.
6. Member is shown a preference prompt — a lightweight modal or interstitial screen — presenting both promoters by display name and avatar with the prompt: 'You have links from two promoters for this venue. Choose who you'd like to support for this event.'
7. Member selects their preferred promoter. The selected promoter becomes the BOUND promoter for this venue+event+date scope. The MemberPromoterPreference record is updated to status = BOUND.
8. All transactions (pre-event bookings, on-night spending, in-app purchases) scoped to that venue+event+date are commissioned to the BOUND promoter, regardless of which link the member originally tapped or any subsequent link taps by the same member for the same scope.
9. The decay counter for the BOUND promoter is set to 0 (binding counts as a code-equivalent attribution event).
10. The non-selected promoter's candidate attribution record is set to status = DECLINED. That promoter receives no commission from this member for this scope.

Binding is Scope-Locked

Once a member binds a promoter for a specific venue+event+date, that binding is IMMUTABLE for the duration of the event window. The member cannot switch their bound promoter after the first transaction has been processed against that scope. This prevents mid-event loyalty switching and protects the promoter who drove conversion.

A.3 Binding Scope Definition

The concept of 'scope' is central to how the multi-promoter preference system works. Scope is the combination of dimensions that define a promoter attribution context:

Scope Type	Binding Applies To	Example
EVENT_DATE	Specific venue + specific event + specific date	Ku De Ta, NYE Gala, 31 Dec 2026
VENUE_DATE	Specific venue + specific calendar date (no event)	Ku De Ta, general Saturday night, 14 Feb 2026
VENUE_OPEN	Specific venue, any date (open-ended link)	Ku De Ta — no date specified in link
PLATFORM_OPEN	No venue or date specified — platform-wide	Generic PRESTIX referral link

EVENT_DATE scope takes highest precedence. If a member has a VENUE_OPEN binding for a venue but also has an EVENT_DATE binding for a specific event at that same venue, the EVENT_DATE binding governs all transactions on that event date.

Precedence order (highest to lowest): EVENT_DATE > VENUE_DATE > VENUE_OPEN > PLATFORM_OPEN.

A.4 Preference UX States

State	Trigger	Commission Outcome	Member Can Change?
NO_CONFLICT	Only one promoter attributed for this scope	Normal attribution / decay model applies	N/A — no conflict
PENDING_CHOICE	Two+ promoter attributions for same scope, no transaction yet	Blocked — no commission processed until choice made	Yes — until first transaction
BOUND	Member selected a promoter for this scope	100% of promoter pool to bound promoter (decay resets)	No — locked after first transaction
BOUND_EXPIRED	Event date has passed	Historical record only — no new commissions	N/A — expired
DECLINED	Non-selected promoter in a resolved conflict	No commission for this scope	N/A — superseded

What if the member never chooses?

If a member enters a PENDING_CHOICE state but proceeds to checkout without resolving the conflict (e.g., by dismissing the prompt), the system defaults to FIRST_ATTRIBUTION: the chronologically first attributed promoter for that scope becomes the BOUND promoter, and the binding is recorded as system-resolved. This prevents checkout abandonment while maintaining an audit trail of how the binding was established.

A.5 Schema — MemberPromoterPreference Model

```
model MemberPromoterPreference {
  id                  String                 @id @default(cuid())
  memberId            String                 // FK → MemberProfile
  member              MemberProfile          @relation(fields: [memberId],
references: [id])

  // Candidate promoters in this scope conflict
  primaryPromoterId   String                 // First attributed promoter
  challengerPromoterId String               // Second attributed promoter
  boundPromoterId     String?               // Null until choice is made

  // Scope — defines the event+venue+date intersection
  bindingScope        PromoterBindingScope   // EVENT_DATE|VENUE_DATE|
VENUE_OPEN|PLATFORM_OPEN
  venueId             String?
  eventId             String?
  scopeDate           DateTime?              @db.Date

  // Resolution
  status              MemberPreferenceStatus // PENDING_CHOICE|BOUND|
BOUND_EXPIRED|DECLINED
  resolutionMethod    String?                // MEMBER_CHOICE |
FIRST_ATTRIBUTION | SYSTEM
  resolvedAt          DateTime?

  // Binding locks after first transaction in scope
  firstTransactionId String?                // FK → Booking; set when binding
locks
  isLocked            Boolean               @default(false)

  createdAt           DateTime              @default(now())
  updatedAt           DateTime              @updatedAt
```

A.6 Commission Resolution at Checkout

The commission pipeline described in the main spec (Section 4) is extended with a pre-step that resolves the effective promoter for any transaction with an active scope:

Step	Check	Outcome
A.PRE-1	Does a MemberPromoterPreference record exist for this member+venue+event+date?	YES → use boundPromoterId as the effective promoter for this transaction. Skip standard attribution lookup.
A.PRE-2	Is the preference status PENDING_CHOICE?	BLOCK checkout. Surface preference prompt. Do not process payment until resolved.
A.PRE-3	Is the preference BOUND?	Use boundPromoterId. Apply decay model against the BOUND promoter's decayCounter (not the global MemberProfile counter). This scope has its own decay counter.
A.PRE-4	No preference record exists	Fall through to standard MemberProfile attribution + decay model as documented in main spec.

Scope-Local Decay Counter

When a BOUND preference exists, the decay counter used for that scope's transactions is stored on the MemberPromoterPreference record itself (boundDecayCounter field), NOT on MemberProfile.decayTransactionCount. This is critical: a member may be at decay counter 3 globally but at counter 0 for a specific bound event-night scope, because they actively chose the promoter for that event. The two counters are independent.

A.7 Extended MemberPromoterPreference Fields

Add the following fields to the model above to support scope-local decay tracking:

Field	Type	Purpose
boundDecayCounter	Int @default(0)	Scope-local decay counter. Starts at 0 when binding is established. Increments on no-code scope transactions.
lastCodeUsedInScope	String?	Last promo code submitted within this scope. Used to verify code-resets within a bound scope.
lastCodeUsedAt	DateTime?	Timestamp of last code submission in this scope.
totalScopeSpend	Decimal(18,2) @default(0)	Running total of all transaction amounts attributed to this bound scope. Used for per-promoter-per-event revenue analytics.
transactionCount	Int @default(0)	Number of completed transactions within this scope. Used for venue performance reporting.

Part B — Venue Promoter Performance Dashboard

The Venue Admin role requires a data-driven view of which promoters are performing well against their venue, enabling informed decisions about ongoing promoter relationships, exclusive arrangements, bonus incentives, and future event promotion mandates. This part defines the metrics, their precise calculation methods, the schema that stores them, and the dashboard surface that presents them.

B.1 Performance Dimensions

The dashboard evaluates promoters across five distinct performance dimensions. Each dimension is independently scored and can be weighted by the venue admin for composite ranking:

Dimension	Code	What It Measures	Data Source
Conversion Rate	CONVERSION	Ratio of members who completed a paid booking or transaction to the total members who clicked the promoter's magic link(s) targeting this venue	MagicLinkClick ÷ (Booking where payment = COMPLETED)
Onboarding Velocity	VELOCITY	Speed at which a member who clicked a link progresses from click → registration → first payment. Measured in hours. Lower = better.	MagicLinkClick.createdAt → MemberProfile.createdAt → BookingPayment.confirmedAt
Pre-Event Booking Rate	PRE_EVENT	Proportion of the promoter's attributed members who make a booking or ticket purchase before the event date (not on the night)	Booking.createdAt < Event.eventDate for promoter's attributed members at this venue
On-Night Spend Rate	ON_NIGHT	Average per-head spend by the promoter's attributed members on the event night, across all in-venue transactions (table additions, bottle service, POS)	Sum(BookingPayment.amount) for on-night transactions ÷ distinct member count
General Venue Spend	GENERAL_SPEND	Total revenue generated at this venue by all members attributed to this promoter, including non-event dates and in-app membership/ transaction purchases	Sum of all BookingPayment amounts at this venue across all attribution types for this promoter

Non-Event Spend Context

The GENERAL_SPEND dimension captures the promoter's contribution to venue revenue outside of specific event nights — regular Saturday visits, membership purchases, and any in-app transactions settled at the venue but not linked to a VenueEvent record. This is the long-tail value of a promoter's network and often the most revealing indicator of true promoter quality.

B.2 Metric Definitions & Calculation Formulae

B.2.1 Conversion Rate

ConversionRate = (UniqueMembers with ≥1 completed booking at venue) ÷ (UniqueMembers who clicked any of the promoter's venue-scoped magic links)

Calculated over a configurable lookback window (default: rolling 90 days). Members who clicked but never registered are included in the denominator. Members who registered but never booked are included in the denominator but not the numerator.

B.2.2 Onboarding Velocity

VelocityScore = Median(time from MagicLinkClick.createdAt to first BookingPayment.confirmedAt) in hours, across all members attributed to this promoter at this venue, within the lookback window.

A lower VelocityScore is better. The dashboard inverts this for display — a promoter with a median 4-hour click-to-payment is scored higher than one with a 72-hour median.

B.2.3 Pre-Event Booking Rate

PreEventRate = (Members who completed a Booking with Booking.isPreEvent = TRUE) ÷ (Total members attributed to this promoter who attended or booked for the event)

Pre-event bookings indicate the promoter's ability to drive advance commitment rather than just walk-in attendance, which is more valuable to the venue for planning and guaranteed revenue.

B.2.4 On-Night Spend Rate

OnNightSpendRate = Sum(all BookingPayment.amount for transactions where transactionContext = ON_NIGHT AND venueId = this venue AND attributedPromoterId = this promoter) ÷ Distinct member count

This captures the average per-head revenue generated on event nights by the promoter's attributed network. High-spending networks are more valuable to venues regardless of raw conversion numbers.

B.2.5 General Venue Spend

GeneralVenueSpend = Sum(all BookingPayment.amount for venueId = this venue AND attributedPromoterId = this promoter AND transactionContext NOT IN [EVENT_TICKET, EVENT_TABLE]) across all time (or configurable lookback)

This is a raw cumulative figure, not a rate. It represents the total AUD value the promoter's network has contributed to the venue outside of specific event nights.

B.3 Composite Performance Score

Each dimension is normalised to a 0–100 score by comparing the promoter's raw metric against the best and worst values observed across all promoters at that venue within the same lookback window. The venue admin can set custom weights per dimension (weights must sum to 100). Default weights:

Dimension	Default Weight	Rationale
CONVERSION	30%	Primary signal — can the promoter actually close bookings?
PRE_EVENT	25%	Advance bookings reduce venue planning risk and guarantee revenue
ON_NIGHT	20%	High per-head spend directly maximises venue revenue per cover
VELOCITY	15%	Fast funnels indicate engaged, intent-ready audiences
GENERAL_SPEND	10%	Long-tail loyalty value — smaller weight to avoid skewing for old promoters

CompositeScore = Σ (NormalisedDimensionScore × DimensionWeight) where NormalisedScore = (PromoterRawValue - MinValue) ÷ (MaxValue - MinValue) × 100

For VELOCITY, the normalisation is inverted: NormalisedScore = (MaxValue - PromoterRawValue) ÷ (MaxValue - MinValue) × 100, since lower velocity time = better performance.

B.4 Dashboard Filter Dimensions

The Venue Admin dashboard must support the following filter and view modes to make the performance data actionable:

Filter	Options	Purpose
Lookback window	7 days / 30 days / 90 days / 6 months / All time	Allows venue to focus on recent performance or view long-term relationships
Event scope	All activity / Specific event / Event series / Non-event dates only	Separates event-night performance from general venue traffic
Promoter status	All / ACTIVE / SUSPENDED / TERMINATED	Ability to review historical data on inactive promoters

Filter	Options	Purpose
Minimum threshold	Min. N members referred / Min. N bookings	Filters out low-volume promoters who skew percentages
Sort by	CompositeScore / Any single dimension / Total revenue / Total bookings	Flexible ranking based on venue's current priority
Tier filter	All / STARTER / BRONZE / SILVER / GOLD / ELITE	Quickly surface top-tier promoters only

B.5 Schema — PromoterVenuePerformance Model

This is the core analytical model storing pre-computed performance metrics per promoter per venue. It is recalculated on a nightly batch job and on-demand when the venue admin refreshes their dashboard.

```
model PromoterVenuePerformance {
  id                    String         @id @default(cuid())
  promoterId            String
  promoter              Promoter       @relation(fields: [promoterId],
references: [id])
  venueId               String
  venue                 Venue          @relation(fields: [venueId],
references: [id])

  // Lookback window for this snapshot
  windowDays            Int            // 7 | 30 | 90 | 180 | 0 (all time)
  snapshotDate          DateTime       @db.Date  // date this was computed

  // Raw metrics
  totalLinkClicks       Int            @default(0)
  totalRegistrations    Int            @default(0)
  totalBookings         Int            @default(0)
  totalCompletedTx      Int            @default(0)
  conversionRate        Decimal        @db.Decimal(6,4)  // 0.0000-1.0000
  medianVelocityHours   Decimal        @db.Decimal(8,2)  // click-to-payment
median
  preEventBookingRate   Decimal        @db.Decimal(6,4)
  onNightSpendPerHead   Decimal        @db.Decimal(18,2)
  generalVenueSpend     Decimal        @db.Decimal(18,2)
  totalCommissionEarned Decimal        @db.Decimal(18,2)

  // Normalised scores (0-100) against all venue promoters
  conversionScore       Int            @default(0)  // 0-100
  velocityScore         Int            @default(0)
  preEventScore         Int            @default(0)
  onNightScore          Int            @default(0)
  generalSpendScore     Int            @default(0)
```

B.6 Schema — PromoterPerformanceSnapshot Model

A time-series record capturing a promoter's per-event performance. One row per promoter per event. Used to power the 'per-event breakdown' view in the venue dashboard and to track trends across an event series.

```
model PromoterPerformanceSnapshot {
  id                    String       @id @default(cuid())
  promoterId            String
  venueId               String
  eventId               String?      // null for non-event-night snapshots
  snapshotType          String       // EVENT_NIGHT | NON_EVENT |
MONTHLY_SUMMARY
  periodStart           DateTime
  periodEnd             DateTime

  // Funnel metrics for this period/event
  linkClicks            Int          @default(0)
  newRegistrations      Int          @default(0)
  preEventBookings      Int          @default(0)
  preEventRevenue       Decimal      @db.Decimal(18,2) @default(0)
  onNightTransactions   Int          @default(0)
  onNightRevenue        Decimal      @db.Decimal(18,2) @default(0)
  commissionsEarned     Decimal      @db.Decimal(18,2) @default(0)
  conversionRate        Decimal      @db.Decimal(6,4)  @default(0)
  avgVelocityHours      Decimal      @db.Decimal(8,2)  @default(0)

  createdAt             DateTime     @default(now())

  @@index([venueId, eventId])
  @@index([promoterId, periodStart])
  @@map("promoter_performance_snapshots")
}
```

B.7 New Enums

```
enum PromoterBindingScope {
  EVENT_DATE      // Bound to specific venue + event + date
  VENUE_DATE      // Bound to venue on a specific calendar date (no event)
  VENUE_OPEN      // Bound to venue, all dates (open-ended)
  PLATFORM_OPEN   // Bound platform-wide, no venue specified
}

enum MemberPreferenceStatus {
  PENDING_CHOICE  // Conflict detected, awaiting member decision
  BOUND           // Member has selected a promoter for this scope
  BOUND_EXPIRED   // Scope period has ended, historical record only
  DECLINED        // Non-selected promoter in resolved conflict
}

enum PromoterPerformanceDimension {
  CONVERSION      // Click-to-booking conversion rate
  VELOCITY        // Median click-to-payment time
  PRE_EVENT       // Pre-event booking rate
  ON_NIGHT        // On-night average spend per head
  GENERAL_SPEND   // Total non-event venue spend
}
```

B.8 VenuePromoterWeightConfig Model

A persistent model storing the venue admin's custom dimension weights. This allows each venue to tune the composite score to reflect their own business priorities (e.g., a nightclub may weight ON_NIGHT heavily; a beach club may weight PRE_EVENT and VELOCITY more).

```
model VenuePromoterWeightConfig {
  id                String   @id @default(cuid())
  venueId           String   @unique
  venue             Venue    @relation(fields: [venueId], references: [id])
  weightConversion Int       @default(30)   // Must sum to 100
  weightPreEvent    Int      @default(25)
  weightOnNight     Int      @default(20)
  weightVelocity    Int      @default(15)
  weightGenSpend    Int      @default(10)
  updatedAt         DateTime @updatedAt
  updatedBy         String   // userId of VENUE_ADMIN who last saved
  @@map("venue_promoter_weight_configs")
}
```

B.9 API Endpoints — Venue Performance Dashboard

Method	Path	Auth	Description
GET	/api/venues/[id]/promoter-performance	VENUE_ADMIN	Return ranked list of all promoters for this venue with all 5 dimension scores and composite score. Supports ?window=30&sort=compositeScore&minBookings=5 query params.
GET	/api/venues/[id]/promoter-performance/[promoterId]	VENUE_ADMIN	Full detail for a single promoter at this venue: all snapshots, funnel breakdown, event-by-event history.
GET	/api/venues/[id]/promoter-performance/[promoterId]/events	VENUE_ADMIN	Per-event snapshot history for a promoter at this venue. Powers the event comparison view.
PUT	/api/venues/[id]/promoter-weight-config	VENUE_ADMIN	Save custom dimension weights. Validates weights sum to 100. Triggers async recomputation of all composite scores for this venue.

Method	Path	Auth	Description
POST	/api/venues/[id]/promoter-performance/refresh	VENUE_ADMIN	On-demand trigger to recompute all PromoterVenuePerformance records for this venue (rate limited: max 1 per hour).

B.10 Dashboard UI Specification

The venue promoter performance dashboard consists of three nested views:

View 1 — Promoter League Table

- Ranked list of all promoters who have generated at least one attribution for this venue
- Columns: Rank | Promoter Name + Tier Badge | Composite Score (0–100) | Conversion Rate | Velocity | Pre-Event Rate | On-Night $/head | Total Revenue Generated
- Sortable by any column. Default sort: CompositeScore descending
- Filter bar: lookback window selector, scope type selector (all / event-night / non-event), tier filter, minimum threshold inputs
- Colour-coded score chips: 80–100 (green), 60–79 (amber), below 60 (red/gray)
- Export to CSV button for offline analysis

View 2 — Promoter Detail Drawer

- Opens on row click. Shows full metric breakdown for the selected promoter.
- Sections: Funnel Overview (clicks → registrations → bookings → payments as a funnel chart), Dimension Scores radar chart, Event History table (per-event snapshot rows), Commission Statement summary
- 'Compare' button to overlay a second promoter's radar chart for side-by-side evaluation

View 3 — Event Night Breakdown

- Accessible from View 2 by clicking a specific event row
- Shows pre-event vs. on-night spend split per promoter for that event
- Attendance confirmation rate (members who booked vs. who checked in via VIP arrival)
- Average time from link click to check-in for that event night

B.11 Batch Computation Job

PromoterVenuePerformance records are maintained by a nightly batch job running at 03:00 venue local time. The job:

11. Queries all venueId+promoterId pairs with activity in the past 180 days.
12. For each pair and each windowDays value [7, 30, 90, 180, 0], computes all 5 raw metrics from source tables (MagicLinkClick, MemberProfile, Booking, BookingPayment).
13. Normalises scores against the current day's min/max across all promoters for the same venue+window.
14. Applies VenuePromoterWeightConfig weights (or defaults if not set) to produce compositeScore.
15. Assigns venueRank (1 = highest compositeScore).
16. Upserts PromoterVenuePerformance rows (insert or replace by unique constraint).
17. Marks any existing rows with the same key but older snapshotDate as isStale = true.

Performance Note

The batch job should process venues serially (not in parallel) to avoid database lock contention on write-heavy tables. For a platform with 50 venues and 200 promoters, the expected per-night computation time is under 2 minutes on a standard Postgres instance with the indexes specified in the schema above.

Summary of All New Schema Additions

Model / Enum	Type	Part	Key Relationships
MemberPromoterPreference	Model	A	MemberProfile, Promoter (×2), Venue, VenueEvent, Booking
PromoterVenuePerformance	Model	B	Promoter, Venue — aggregated, no FK to individual bookings
PromoterPerformanceSnapshot	Model	B	Promoter, Venue, VenueEvent — time-series event rows
VenuePromoterWeightConfig	Model	B	Venue (1:1) — venue admin weight preferences
PromoterBindingScope	Enum	A	Used on MemberPromoterPreference.bindingScope
MemberPreferenceStatus	Enum	A	Used on MemberPromoterPreference.status
PromoterPerformanceDimension	Enum	B	Used in API filter params and weight config

Open Design Questions for Stakeholder Alignment

#	Question	Options	Recommendation
1	Should a member be able to change their bound promoter for a scope BEFORE any transaction, even after explicitly choosing?	A) Allow change until first tx B) Lock immediately on choice	A — Allow change until first transaction. UX flexibility without financial risk since no money has moved yet.
2	What happens if the BOUND promoter is SUSPENDED after binding but before the event?	A) Rebind to platform (100% platform) B) Offer member new preference prompt	B — Surface a new preference prompt. Forced platform-only allocation penalises the member for the platform's decision.
3	Should the venue see member-level granularity (which specific members came via which promoter)?	A) Yes, full member list B) Aggregated only (counts and spend, no names)	B — Aggregate only by default. Member-level data is available to PLATFORM_ADMIN for support purposes only, protected by access policy.
4	Should promoter performance scores be visible to promoters themselves?	A) Yes — full transparency B) Partial (their own scores, not competitors') C) No	B — Promoters see their own scores and their venue rank (e.g. '#3 of 12 at Ku De Ta') but not competitor details.

PRESTIX.VIP — Addendum B | Confidential Technical Specification | Version 1.0 | March 2026